RÉSUMÉ : SERVICES WEB REST

Partie 1/3 — Le style d'architecture REST

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 3 · 2026-2027

1 QU'EST-CE QUE REST ?

REST (REpresentational State Transfer) : **style d'architecture** inspiré du fonctionnement du Web, défini par Roy Fielding dans sa thèse de doctorat (2000). Une application qui le respecte est dite **RESTful**.

Fielding est co-auteur de la spécification HTTP et co-fondateur du projet Apache HTTP Server.

Style d'architecture : ensemble de **contraintes** qui, appliquées aux composants d'un système, optimisent certains critères (performance, simplicité, évolutivité…).

2 REST EST… / N'EST PAS…

REST est…
- un style d'architecture, non standardisé
- une approche pour construire une application
- fondé sur des standards : HTTP, URI, XML / JSON

REST n'est pas…
- un format de données
- un protocole de communication
- un standard officiel (contrairement à SOAP)

3 POURQUOI REST ? (ALTERNATIVE À SOAP)

Léger et simple
messages courts, faciles à produire et à décoder

URI lisibles
navigation naturelle entre ressources

Sans état (stateless)
pas de session côté serveur → moins de mémoire

Cache-friendly
réponses cacheables → meilleure montée en charge

4 LES 6 CONTRAINTES DE REST

1 **Client-serveur** : interface séparée des données et de la logique

2 **Sans état** : chaque requête contient tout ce qu'il faut

3 **Cache** : les réponses indiquent si elles sont cacheables

4 **Interface uniforme** : URI, représentations, messages auto-descriptifs, hypermédia

5 **Système en couches** : le client ignore les intermédiaires (proxy, cache)

6 **Code à la demande** (optionnelle) : le serveur envoie du code (ex. JavaScript)

Attention : utiliser HTTP et de belles URI ne suffit pas ; une API est RESTful si elle respecte ces contraintes.

5 EXEMPLES D'ÉCHANGES : GET ET POST



6 ARCHITECTURE RESTFUL

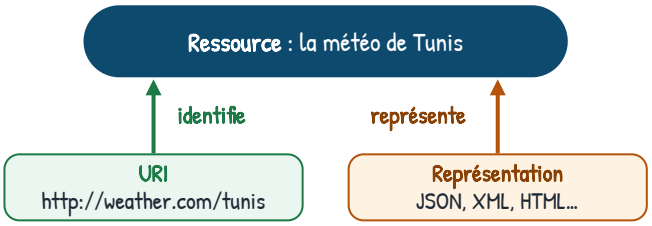


→ requêtes HTTP : **GET, POST, PUT, DELETE** · ← réponses : **JSON / XML**

Client : sérialise la requête (paramètres, corps), puis parse la réponse.
Serveur : reçoit l'appel, exécute le traitement, sérialise la réponse au format attendu.

→ HTTP transporte les messages ; un parseur (JSON, XML…) traduit les données de chaque côté.

7 RESSOURCE, URI, REPRÉSENTATION



Ressource : toute entité identifiable (un livre, un étudiant…).

URI : son identifiant unique, ex. http://localhost:8080/libraryrestwebservice/books

Représentation : vue de l'état de la ressource, transférée entre client et serveur (JSON, XML, texte…).

Méthodes : les verbes HTTP manipulent la ressource.

8 LA REPRÉSENTATION DES RESSOURCES

GET (réponse) : format de **sortie** attendu — JSON, XML, texte, HTML…

POST / PUT (corps) : format d'**entrée** envoyé — JSON, XML…

```
{
 "id": 12,
 "nom": "Dupont",
 "actif": true
}
```

→ Le format de sortie et le format d'entrée d'une même ressource peuvent différer.

RÉSUMÉ : SERVICES WEB REST

Partie 2/3 — Concevoir une API RESTful

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 3 · 2026-2027

9 VERBES HTTP ET CRUD

Verbe	CRUD	Effet	Sûr	Idemp.
GET	Read	Lire une ressource / collection	Oui	Oui
POST	Create	Créer (sous une collection)	Non	Non
PUT	Update	Remplacer intégralement	Non	Oui
PATCH	Update	Modifier partiellement	Non	Non*
DELETE	Delete	Supprimer	Non	Oui

Sûr (safe) : ne modifie pas l'état du serveur. **Idempotent** : plusieurs appels identiques ont le même effet qu'un seul.

* PATCH n'est idempotent que si son opération l'est (ex. remplacer la valeur d'un champ).

10 NOMMER LES RESSOURCES

Règle d'or : des **noms** dans l'URI, jamais de verbes ; l'action est portée par le **verbe HTTP**.

- Pluriel pour les collections : `/books` plutôt que `/book`
- Minuscules et kebab-case : `/order-items`
- Une seule convention dans toute l'API

À éviter ✗	Recommandé ✓
`GET /getAllBooks`	`GET /books`
`POST /createBook`	`POST /books`
`GET /book/1/delete`	`DELETE /books/1`
`GET /Books_List`	`GET /books`

11 HIÉRARCHIE ENTRE RESSOURCES

Les relations logiques se reflètent dans la structure de l'URI :

```
GET /clients/12/commandes
→ la liste des commandes du client 12
GET /clients/12/commandes/45
→ la commande 45 du client 12
```

Limite : 2 à 3 niveaux au maximum ; au-delà, préférer des liens ou des paramètres de requête.

12 LES BONS CODES DE STATUT

`200 OK`	**Succès** — GET réussi
`201 Created`	**Ressource créée** — POST réussi
`204 No Content`	**Succès sans corps** — DELETE réussi
`400 Bad Request`	**Requête invalide** — champ manquant, mal formé
`401 Unauthorized`	**Authentification requise** — jeton absent ou invalide
`404 Not Found`	**Ressource introuvable** — identifiant inexistant
`409 Conflict`	**Conflit d'état** — doublon, version obsolète
`500 Internal…`	**Erreur serveur** — exception non gérée

13 FILTRER, TRIER, PAGINER : LES PARAMÈTRES DE REQUÊTE

Filtrer
Restreindre une collection selon un ou plusieurs critères.
`GET /books?author=camus`

Trier
Ordonner les résultats (préfixe « - » = ordre décroissant).
`GET /books?sort=publishedAt`

Paginer
Limiter le nombre de résultats renvoyés par page.
`GET /books?page=2&size=20`

→ Ces opérations sélectionnent une collection : toujours en paramètres de requête, jamais dans le chemin.

14 VERSIONNER SON API

Dans l'URI
`GET /v1/books`
+ simple, visible · – « pollue » l'identifiant

Dans un en-tête HTTP
`Accept: application/vnd.api.v1+json`
+ URI stable · – client à configurer

En paramètre de requête
`GET /books?version=1`
+ facile à tester · – moins conventionnel

→ Objectif : faire évoluer l'API sans casser les clients existants.

15 CHECK-LIST D'UN ENDPOINT REST

- ✓ Ressource nommée par un substantif au pluriel
- ✓ Aucun verbe d'action dans l'URI
- ✓ Verbe HTTP cohérent avec l'opération
- ✓ Hiérarchie d'URI limitée et lisible
- ✓ Filtrage, tri, pagination en paramètres de requête
- ✓ Codes de statut précis et cohérents
- ✓ Stratégie de versionnement définie dès la conception
- ✓ Réponses au format cohérent (JSON par défaut) et documentées

RÉSUMÉ : SERVICES WEB REST

Partie 3/3 — Implémentation avec JAX-RS

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 3 · 2026-2027

16 DÉVELOPPER ET DOCUMENTER UN SERVICE REST

1 Concevoir ressources et endpoints › 2 Implémenter (ex. JAX-RS) › 3 Compiler, déployer, tester › 4 Documenter (OpenAPI, WADL)

WADL (Web Application Description Language) : description XML de services REST, soumise au W3C par Sun en 2009 (simple *Member Submission*). Peu outillée, elle est remplacée en pratique par **OpenAPI / Swagger**.

17 JAX-RS

JAX-RS (Java API for RESTful Web Services) : spécification Java pour mettre en œuvre et consommer des services REST, fondée sur des **annotations**.

Implémentations : Jersey · Apache CXF · RESTEasy · Restlet

Approche du cours : bottom-up — annoter une classe POJO, puis compiler et déployer. Le service n'est pas généré à partir d'un WADL.

Packages : `javax.ws.rs` (Java EE) ; `jakarta.ws.rs` depuis Jakarta EE 9.

18 LES ANNOTATIONS JAX-RS

`@ApplicationPath`	Racine de toutes les ressources ; sur une sous-classe de **Application**
`@Path`	URI relative ; sur la classe et/ou la méthode (les deux se **concatènent**)
`@GET @POST @PUT @DELETE`	Lie une méthode à un verbe HTTP ; le nom de la méthode n'a pas d'importance
`@Produces`	Type de la **réponse** (ex. MediaType.APPLICATION_JSON)
`@Consumes`	Type accepté **en entrée** (corps de la requête)

19 COMMENT SE CONSTRUIT L'URL

`http://localhost:8080/Bibliotheque/webresources/category/hello/Miage`

`http://localhost:8080`	serveur + port
`/Bibliotheque`	contexte de l'application
`/webresources`	**@ApplicationPath**
`/category`	**@Path** de la classe
`/hello/Miage`	**@Path** de la méthode : `"hello/{nom}"`

20 RÉCUPÉRER LES PARAMÈTRES

Ces annotations injectent des valeurs de la requête dans les paramètres de la méthode :

`@PathParam`	segment variable {…} du chemin — `/hello/Miage`
`@QueryParam`	paramètre après « ? » — `?from=Sarra&to=Celine`
`@FormParam` `@HeaderParam` `@CookieParam`	champ de formulaire, en-tête HTTP, cookie
`@DefaultValue`	valeur par défaut si la clé est absente

21 EXEMPLE : UNE RESSOURCE CRUD /BOOKS

```
@Path("books")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class BookResource {
  @GET
  public List<Book> getAll() {…}
  @GET @Path("{id}")
  public Book getById(@PathParam("id") int id) {…}
  @POST
  public Response create(Book b) {…}
  @PUT @Path("{id}")
  public Book update(@PathParam("id") int id, Book b) {…}
  @DELETE @Path("{id}")
  public Response delete(@PathParam("id") int id) {…}
}
```

URI	Verbe	Action → statut
/books	GET	liste des livres → 200
/books	POST	créer un livre → 201
/books/{id}	GET	un livre → 200 / 404
/books/{id}	PUT	mettre à jour → 200
/books/{id}	DELETE	supprimer → 204

@Produces / @Consumes sur la classe s'appliquent à toutes ses méthodes.

Response permet de fixer le code de statut (201, 204, 404…).

```
@ApplicationPath("webresources")
public class BibliothequeApp
    extends Application { }
```

POINTS À RETENIR

Une ressource = une **URI** ; l'action = le **verbe HTTP** ; le résultat = le **code de statut**.

RESTful = respecter les contraintes de REST (sans état, interface uniforme…), pas seulement utiliser HTTP.

JAX-RS : @Path + verbe + @Produces / @Consumes ; @PathParam pour {…}, @QueryParam après « ? ».